Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I expect to use for loops, while loops, input boxes, and dataclass instances in this assignment. I want to improve my skills with for loops, as that is something I have previously struggled with.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect that the easiest part of this project to be the implementation of simple helper functions into my code, as that is something I do well. However, I expect that the most difficult part will be finding different routes to use to improve my design, as there are many routes required.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

The tasks that will require the most time and effort will likely be the construction and connection of the different routes and pages, as this is a long and complex code that will require lengthy routes and pages. The features that should be implemented first are the base page of the website and the underlying code to connect the functions.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Other classwork may interfere with my work, which I can minimize the impact of by staying on top of my work and prioritizing this project. The office hours will be a great resource while working on this project. I do not have any questions about the assignemnt requirements.

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

CISC106 Flowchart 11/05

State:

- difficulty: str
- current_round: int
- total_rounds: int
- streak: int
- score: int
- timer_limit: int
- questions: list[Question]

PAGES:

home_page()

Game Title

Description of game

[Start Game]

[Instructions]

[Leaderboard]

leaderboard_page()

LEADERBOARD

Highest Score: __

Largest Streak: __

Fastest average time: __

[Back]

instructions_page()

Instructions (text displaying how game is played)

[Back]

settings_page()

Difficulty:

(Easy)(Medium)(Hard)

Number of rounds:

(5)(10)(15)

Timer Length:

(5sec)(10 sec)(15 sec)

[Start] [Back]

game_page()

Round = __/10

Question:

__ + __ = ?

Answer: ____

Timer: __ seconds

Score: __

Streak: __

[Submit] [Quit]

result_page()

Correct

or

Incorrect

or

Time's up

Correct answer: __

Score: __

Streak: __

[Next Question]

[Finish Game]

summary_page()

GAME OVER

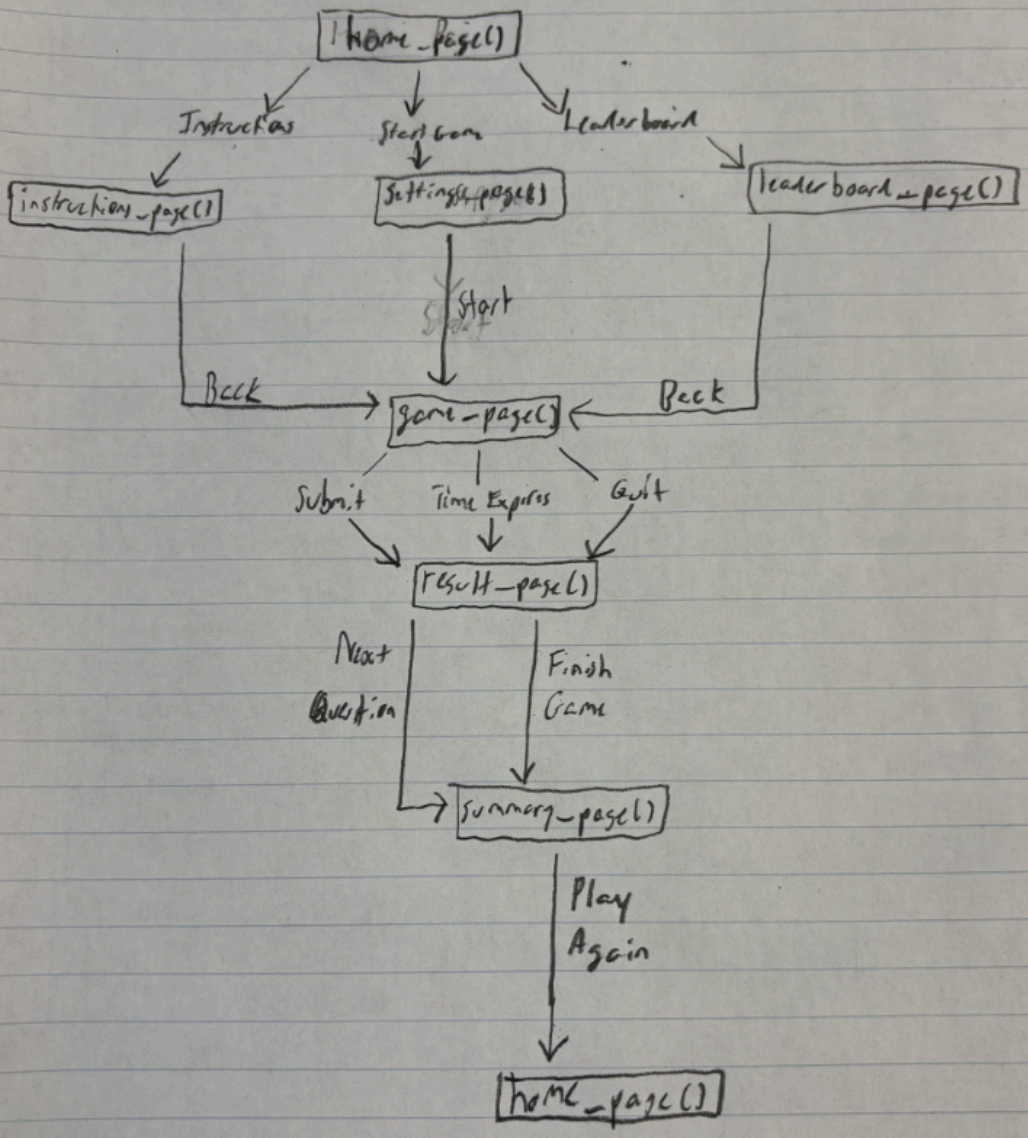
Final Score: __

Longest Streak: __

Accuracy: __

[Play Again

CISC 106 Flow Chart 11/05



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**The purpose of this site is to help people practice their quick mathematics skills in a fun and unique way.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	- difficulty : that represents the selected difficulty by the user, which determines the questions generated. Difficulty can be updated after each game. Type is str. - current_round: tells the user and tracks what round they are currently on. Type is int. Will be modified by increasing by 1 every time the user progresses to a new round. - total_rounds: stores the number of total rounds desired by the user. Type is int. Will be modified each time the user selects the number of rounds, or questions, that they want. - streak: stores how many consecutive correct answers the user gets, and resets after an incorrect answer. Type is int. Will be modified every time the user gets another consecutive correct answer. - score: stores the user's total score as they progress through the game. Type is int. Will be modified every time a question is answered

	correctly by the user. - timer_limit: stores the number of seconds that the user selects as the timer for each question. Type is int. Will be modified every time the user selects a different timer limit at the start of the game. - questions: this attribute stores all of the questions that were generated and answered during the game. Has a list of another dataclass. Type is list[Question]. Will be modified after each game is played, as new questions will be asked.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Dataclass will be defined as Question, with 5 attributes - prompt (str): stores the question that is displayed to the user, and is modified each time a new question is prompted to the user. - answer (int): stores the correct answer for that prompt, and will be modified each time a new prompt is displayed to the user with a new correct answer. - user_answer (str): stores the user's answer to the prompt, and will be modified each time the user inputs their answer. - correct (bool): compares the answer to user_answer and stores whether the comparison is True (correct) or False (incorrect). Modified each time a new prompt is displayed. - time_used (int): stores how much time the user had when they submitted their response, or if time ran out before submission. Modified

	after each new prompt is answered by the user or if time runs out.
Routes (minimum 7) The names of your route functions	- home_page() - instructions_page() - settings_page() - game_page() - result_page() - summary_page() - leaderboard_page()
Pages (minimum 5) A brief description of the pages in your site	- Home Page: Starting page of the game, has the game title, the instructions, and the Start button - Instructions Page: tells the user how the game is supposed to be played, including the purpose and a complete ruleset - Settings Page: lets the user customize their game. Has three user inputs, the DifficultySelection, which lets them choose their difficulty, NumberOfRounds, which lets them select how many rounds they want, and TimerLength, which lets them choose how much time per question they want, also has a Start button - Game Page: where the game is played. Displays the question, an input box, the score and streak counter, the round number, the

	countdown timer, and takes input in the input box when the user answers the question - Result Page: displays whether or not the last question was correct, incorrect, or if the user ran out of time, and displays the updated score and streak counts - Summary page: displays the summary of the game for the user after the game is over, including their final score and highest streak, rounds completed, and accuracy precentage - Leaderboard Page: displays the highest score since the game has started, the longest streak, and info from previous games. This would be accessed from the Home page
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	- DifficultySelection: lets the user choose their desired difficulty for the math problems, choices of easy, medium, and hard - NumberOfRounds: lets the user choose how many rounds of questions they want, multiple choice options - TimerLength: lets the user choose how much time they want to answer each question, multiple choice options - MathAnswer: the user inputs their response to the math prompt in a text box that will be evaluated for correctness
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	- Difficulty selection: If statement that will generate an easy question if “easy” difficulty is selected, a medium question if “medium” difficulty is selected, and a hard question if “hard” difficulty is selected. This is meaningful because it allows the user to choose their

	desired difficulty and directs them to the correct page. - Correct answer checker: this if statement compares the MathAnswer input to the correct answer, and will increase the state.streak instance if the input is equal to the correct answer. If it is not, the state.streak equals 0. This is meaningful because it checks for correctness, which is essential to the function of the game - Time expiration: This if statement checks to see if the timer has expired. If it has, then the question.correct instance will be False, and the streak will be reset to 0. This is important because it resets the streak if the user runs out of time before they input an answer - End of game: this if statement checks to see if the user has made it through all of the rounds, if the state.current.round == state.total.rounds, then the game is over and the user is directed to the summary page
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	This loop iterates over the state.questions instance, and keeps count of the consecutive correct answers given. This is essentially a streak counter. For every question answered correctly in a row, 1 point will be added to the streak counter, which resets after an incorrect answer. This is meaningful to the site because it provides an interactive way for the user to stay engaged.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

The purpose of this site is to help people practice their quick mathematics skills in a fun and unique way. Based on our previous chat, the drafter game should include different options for difficulty to challenge all user in mathematic operations, include a streak counter that ends when the answer is wrong, several rounds, and if possible a timed componet. guide me to making a flowchart that will include all of the following: We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages. In addition to the pages and routes, write down what fields your `State` will have, both their names and types. there must be a state dataclass with at least 4 attributes, one which is a list/dictionary of the second dataclass. 7 routes and 5 pages minimum, with 3 unique user inputs, 3 if statements and a loop that iterates over the list/dictionary of dataclass instances in the state attribute

For your Drafter Game (Quick Math Practice) project, you need to show enough detail to satisfy the CISC 106 requirements: 5+ pages, 7+ routes, a State dataclass with 4+ attributes, a second dataclass stored in a list/dictionary, 3 user inputs, 3 if statements, and a loop over the dataclass list/dictionary.

State Design

Dataclass #1: State

```
@dataclass
class State:
    difficulty: str
    current_round: int
    streak: int
    score: int
    time_left: int
    questions: list[Question]
```

Dataclass #2: Question

```
@dataclass
class Question:
    prompt: str
    answer: int
    user_answer: str
    correct: bool
```

State Attributes and Types

Attribute	Type	Purpose
difficulty	str	Easy, Medium, Hard
current_round	nt	Current round number
streak	nt	Consecutive correct answers
score	nt	Total points
ime_left	nt	Countdown timer
questions	ist[Question]	Stores all question objects

This satisfies the requirement of having a State dataclass with at least 4 attributes and a list of another dataclass.

User Inputs

You need at least 3 unique inputs.

Input 1

Difficulty Selection

Easy

Medium

Hard

Input 2

Math Answer

User types numerical answer

Input 3

Number of Rounds

5

10

15

Pages / Routes

You need at least 5 pages and 7 routes.

Page 1: home_page()

Contents

- Title: Math Drafter
- Start Game Button
- Instructions Button

Routes

`Start Game → difficulty_page`

`Instructions → instructions_page`

Page 2: instructions_page()

Contents

- Game explanation
- Difficulty descriptions
- Timer explanation
- Back button

Route

`Back → home_page`

Page 3: difficulty_page()

Inputs

- Difficulty dropdown
- Number of rounds input

Buttons

- Continue
- Back

Routes

`Continue → game_page`

`Back → home_page`

Parameters Passed

`difficulty`

`round_count`

Page 4: game_page()

Contents

- Current question
- Answer input box
- Streak counter
- Round number
- Timer

User Input

`answer`

Routes

`Submit Answer → result_page`

`Quit Game → summary_page`

Parameters Passed

question

difficulty

streak

score

current_round

Page 5: result_page()

Contents

- Correct/Incorrect message
- Updated score
- Updated streak

Routes

Plain Text

Next Question → game_page

Finish Game → summary_page

Page 6: summary_page()

Contents

- Final Score
- Longest Streak
- Accuracy

- Play Again Button

Routes

Plain Text

Play Again → home_page

Route Diagram

You can draw arrows like this:

Plain Text

+-------------+

| home_page |

+-------------+

|

Start Game

v

+-------------------+

| difficulty_page |

+-------------------+

| Continue

v

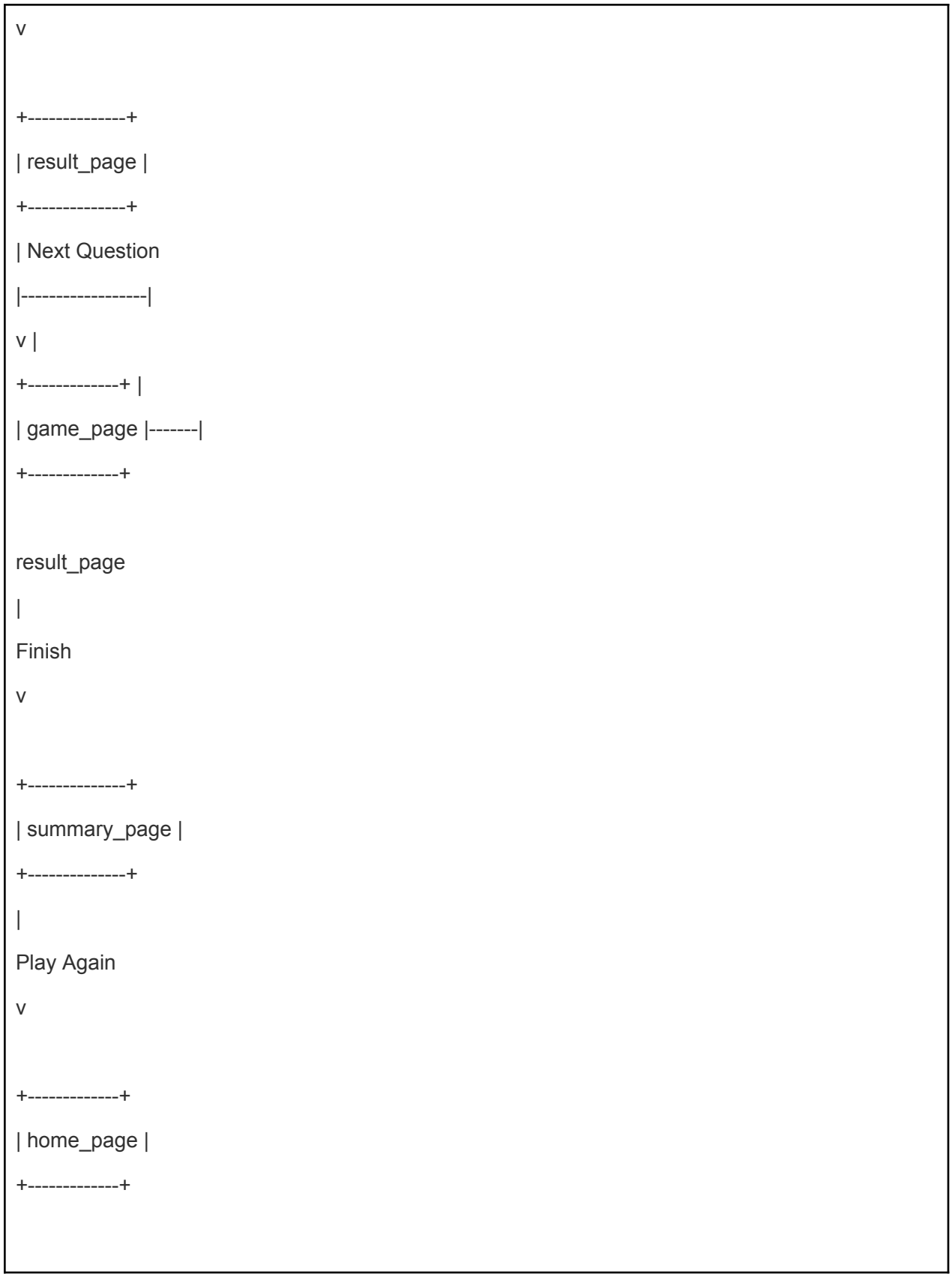
+-------------+

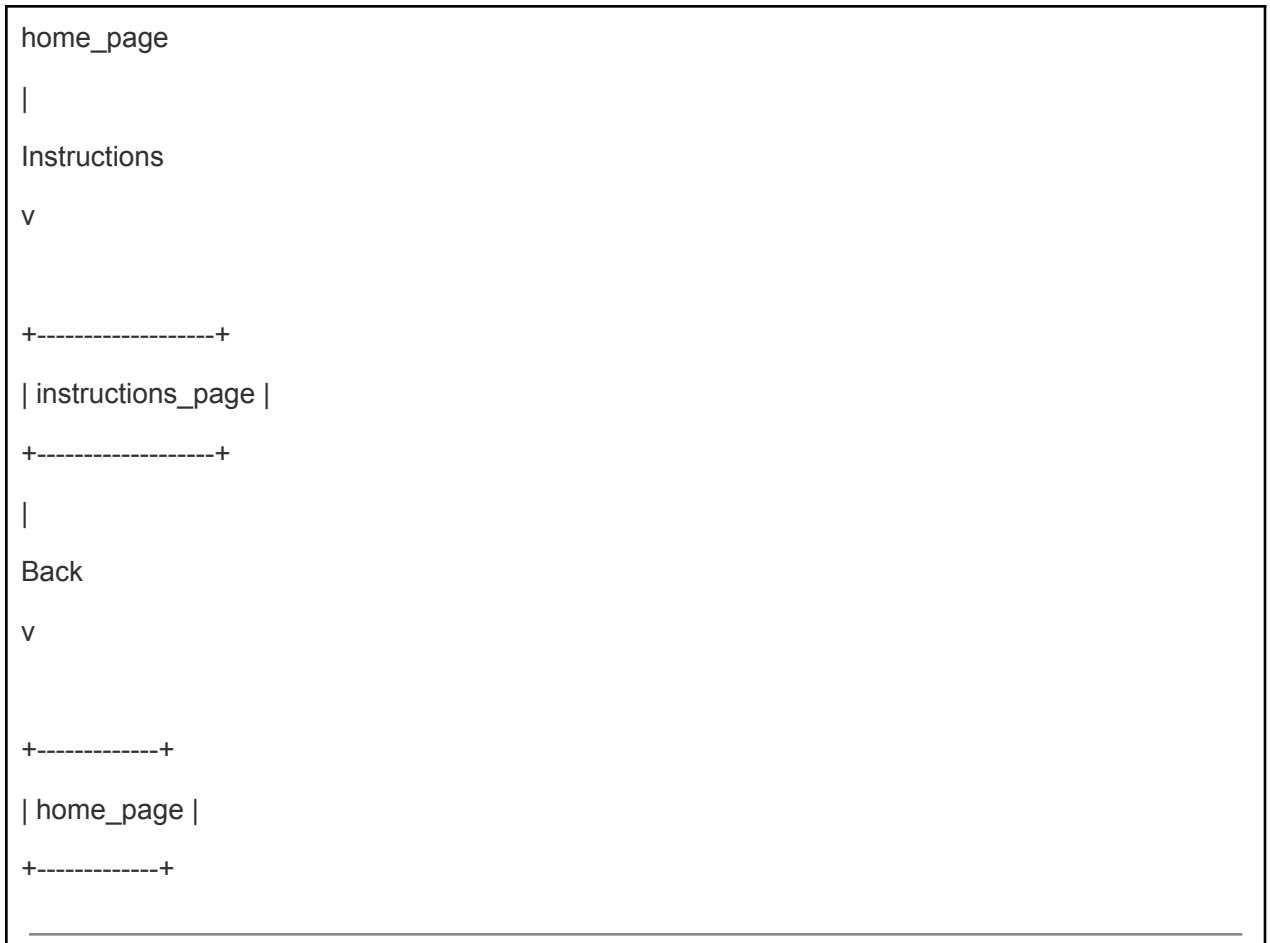
| game_page |

+-------------+

|

Submit Answer


v
+--------------+
| result_page |
+--------------+
| Next Question
|-----------------|
v |
+-------------+ |
| game_page |-------|
+-------------+
result_page
|
Finish
v
+--------------+
| summary_page |
+--------------+
|
Play Again
v
+-------------+
| home_page |
+-------------+



Seven Routes Checklist

1. home_page → difficulty_page
2. home_page → instructions_page
3. instructions_page → home_page
4. difficulty_page → game_page
5. difficulty_page → home_page
6. game_page → result_page
7. game_page → summary_page
8. result_page → game_page
9. result_page → summary_page
10. summary_page → home_page

This exceeds the required 7 routes.

Required If Statements

If #1 Difficulty Selection

Python

if state.difficulty == "Easy":

generate_easy_problem()

elif state.difficulty == "Medium":

generate_medium_problem()

else:

generate_hard_problem()

If #2 Answer Checking

Python

if user_answer == question.answer:

state.score += 10

state.streak += 1

else:

state.streak = 0

If #3 Timer Check

Python

if state.time_left <= 0:

goto(summary_page)

Required Loop Over Dataclass List

Since `questions` is a list of `Question` objects:

Python

correct_count = 0

for question in state.questions:

if question.correct:

correct_count += 1

This satisfies the requirement of **iterating over the list of dataclass instances stored in State**.

What to Put on Your Flowchart

Label each rectangle with the function names:

Plain Text

home_page()

instructions_page()

difficulty_page()

game_page()

result_page()

summary_page()

Write inside the pages:

- Text displayed
- Buttons
- Inputs

Write on arrows:

Plain Text

Start Game

Instructions

Back

Continue

Submit Answer

Next Question

Finish Game

Play Again

Include parameter notes near arrows, such as:

Plain Text

difficulty

round_count

score

streak

question

This design fully meets the CISC 106 project requirements while matching your math game idea of difficulty levels, streak tracking, multiple rounds, and an optional timer.

is it possible to have a timed componet to this, where the user must type the numerical answer beforer the timer runs out, or else the answer is incorrect?

Yes, and it would actually make your game more interesting while still fitting the CISC 106 requirements.

You could add a timer for each question so that if the player does not enter an answer before the timer reaches 0, the question is automatically marked incorrect and their streak resets.

Updated State Dataclass

```
@dataclass
```

```
class State:

   difficulty: str

   current_round: int

   streak: int

   score: int

   time_remaining: int

   questions: list[Question]
```

Additional User Input

You could even let the user choose a timer setting:

```
5 seconds

10 seconds

15 seconds
```

This would become a fourth user input.

Timer Logic

When a question appears:

```
state.time_remaining = 10
```

The timer counts down.

When the player submits an answer:

```
if state.time_remaining > 0:
   check_answer()
else:
   mark_incorrect()
```

Required If Statement

This gives you another clear if statement:

```
if state.time_remaining <= 0:

   state.streak = 0

   question.correct = False

   go_to(result_page)
```

Flowchart Changes

game_page()

Add:

```
Question

Answer Box

Timer

Score

Streak

Round Number
```

Inside the page rectangle, you might note:

```
Countdown Timer

10 → 9 → 8 → ...
```

New Route

You could show an additional arrow from Game Page to Result Page:

```
Timer Expires
```

Example:

```
+-------------+

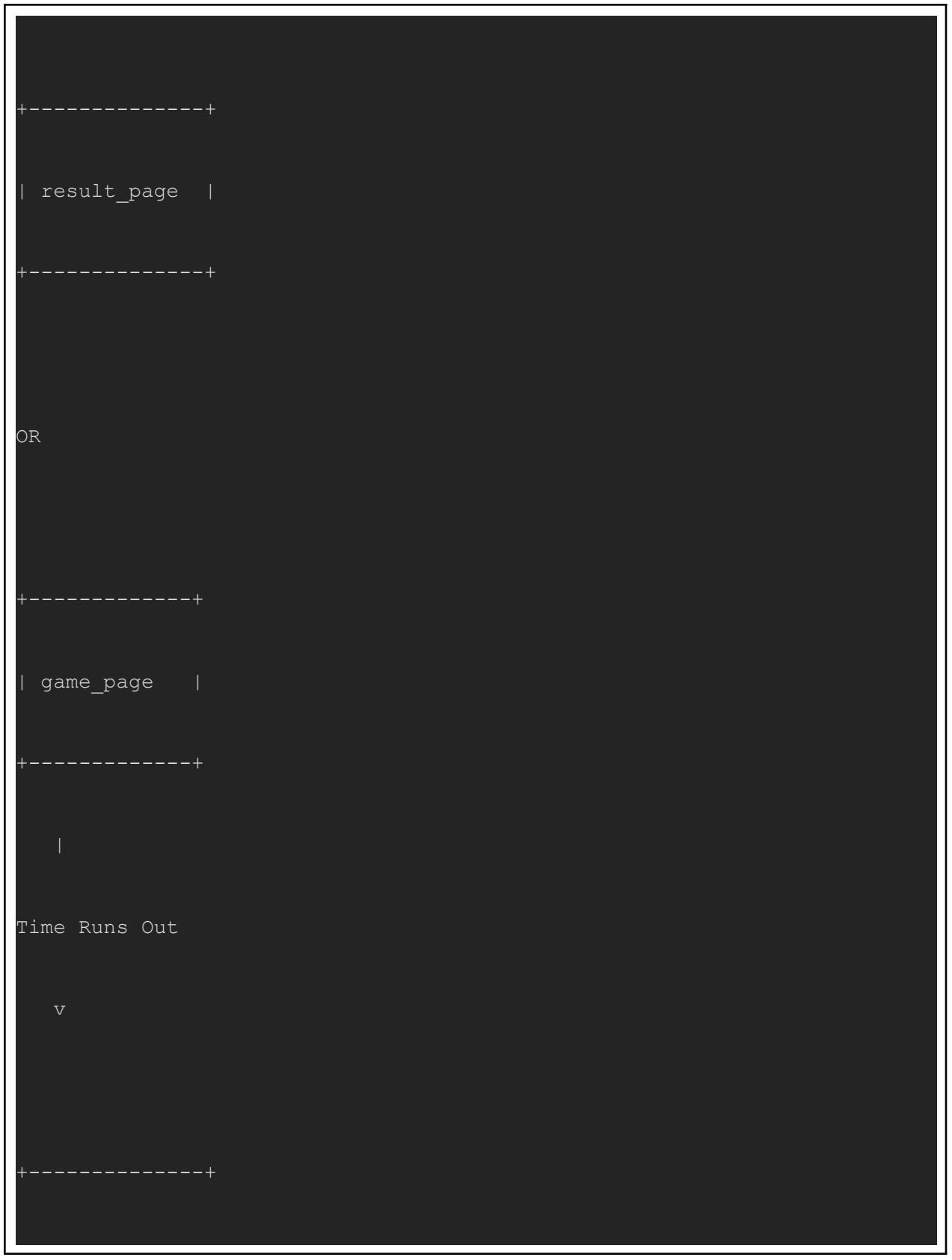
| game_page   |

+-------------+

   |

Submit Answer

   v
```


+--------------+
| result_page |
+--------------+
OR
+--------------+
| game_page |
+--------------+
|
Time Runs Out
v
+--------------+

```
| result_page  |

+--------------+
```

Difficulty-Based Timers (Optional)

This would add another use of your difficulty setting:

```
if state.difficulty == "Easy":

   state.time_remaining = 15



elif state.difficulty == "Medium":

   state.time_remaining = 10



else:

   state.time_remaining = 5
```

That makes the game progressively harder:

Difficulty	Time Limit
Easy	15 sec
Medium	10 sec
Hard	5 sec

Why This Is Good for the Project

Adding the timer helps you satisfy several requirements naturally:

- ✅ More user interaction
- ✅ Additional if statement (`timer <= 0`)
- ✅ More state attributes (`time_remaining`)
- ✅ Additional route (`Time Runs Out`)
- ✅ Makes the streak counter more meaningful

- ✅ Fits the purpose of practicing quick mathematics skills

For your flowchart, I would definitely include the timer because it supports the project's goal of testing both math accuracy and speed.

cisc 106 project

update the guided flowchart and all of the requirements with this timer implemented

Yes. With the timer implemented, your project is actually stronger because it emphasizes quick mathematics skills, which matches your project's purpose statement.

Project Requirements Checklist

Pages

✅ 6 Pages (minimum is 5)

1. `home_page()`
2. `instructions_page()`
3. `settings_page()`
4. `game_page()`
5. `result_page()`
6. `summary_page()`

Routes

✅ 10 Routes (minimum is 7)

1. Home → Instructions
2. Instructions → Home
3. Home → Settings
4. Settings → Home
5. Settings → Game
6. Game → Result (Submit Answer)
7. Game → Result (Timer Expires)
8. Result → Game
9. Result → Summary
10. Summary → Home

User Inputs

✅ 4 Inputs (minimum is 3)

1. Difficulty Selection
2. Number of Rounds
3. Timer Length

4. Math Answer

If Statements

✅ 4 If Statements (minimum is 3)

1. Difficulty determines question complexity.
2. Correct vs incorrect answer.
3. Timer expires before answer.
4. Last round reached.

Loop

✅ Loop through list of Question dataclass objects.

Dataclasses

✅ State dataclass with 6 attributes.

✅ Question dataclass stored in a list inside State.

Dataclass Design

Question Dataclass

```
@dataclass

class Question:

   prompt: str

   answer: int

   user_answer: str

   correct: bool

   time_used: int
```

State Dataclass

```
@dataclass
```

```
class State:

   difficulty: str

   current_round: int

   total_rounds: int

   streak: int

   score: int

   timer_limit: int

   questions: list[Question]
```

State Attributes

Attribute	Type	Purpose
difficulty	str	Easy, Medium, Hard

current_round	nt	Current question number
otal_rounds	nt	Number of rounds selected
streak	nt	Consecutive correct answers
score	nt	Total points
imer_limit	nt	Seconds allowed per question
questions	ist[Question]	Stores all question objects

Page 1: home_page()

Components

Math Drafter

Quick Math Challenge

[Start Game]

[Instructions]

Routes

Start Game → settings_page()

Instructions → instructions_page()

Page 2: instructions_page()

Components

How To Play:

```
1. Choose a difficulty.


2. Choose rounds.


3. Solve each problem before time expires.


4. Wrong answers OR timeout reset streak.


5. Highest score wins.




[Back]
```

Route

```
Back → home_page()
```

Page 3: settings_page()

User Inputs

Difficulty

```
Easy


Medium
```

```
Hard
```

Number of Rounds

```
5


10


15
```

Timer

```
5 seconds


10 seconds


15 seconds
```

Buttons

```
Start


Back
```

Routes

```
Start → game_page()




Back → home_page()
```

Parameters Passed

difficulty

total_rounds

timer_limit

Page 4: game_page()

Components

Round 3 / 10

Question:

8 × 7 = ?

Answer Box

Timer: 7 seconds

```
Score: 20




Current Streak: 2




[Submit]


[Quit]
```

User Input

```
answer
```

Routes

Submit Button

```
Submit → result_page()
```

Quit Button

```
Quit → summary_page()
```

Automatic Timeout Route

```
Timer Reaches 0


↓


result_page()
```

Parameters Passed

```
question


score


streak


current_round


timer_limit
```

Page 5: result_page()

Components

If Correct:

Plain Text

✅ Correct!

Score +10

Streak: 4

If Incorrect:

Plain Text

❌ Incorrect

Correct Answer: 56

Streak Reset

If Timeout:

Plain Text

⏰ Time's Up!

Correct Answer: 56

Streak Reset

Buttons

Plain Text

Next Question

Finish Game

Routes

Plain Text

Next Question → game_page()

Finish Game → summary_page()

Page 6: summary_page()

Components

Plain Text

Game Over

Final Score: 85

Longest Streak: 6

Questions Correct: 8/10

Accuracy: 80%

[Play Again]

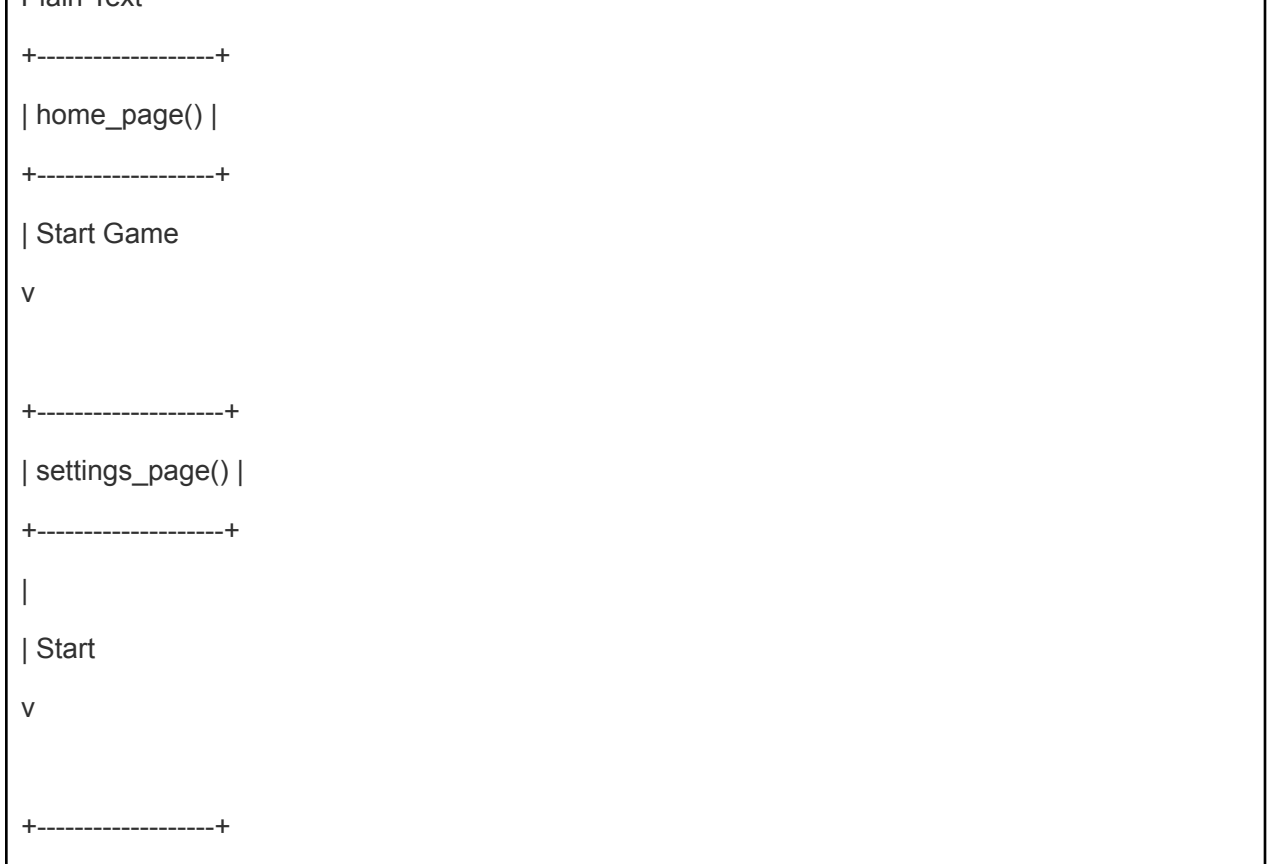
Route

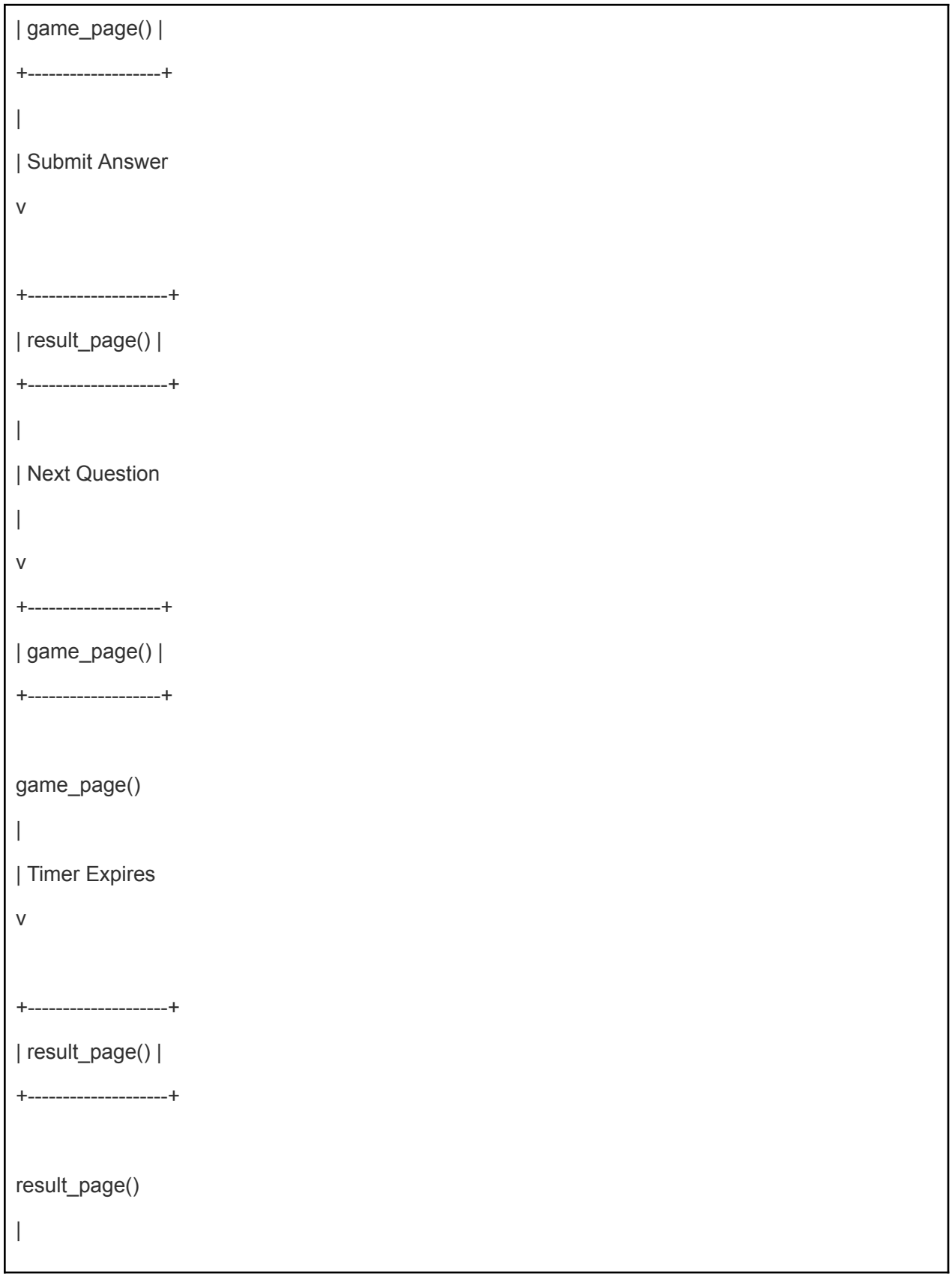
Plain Text

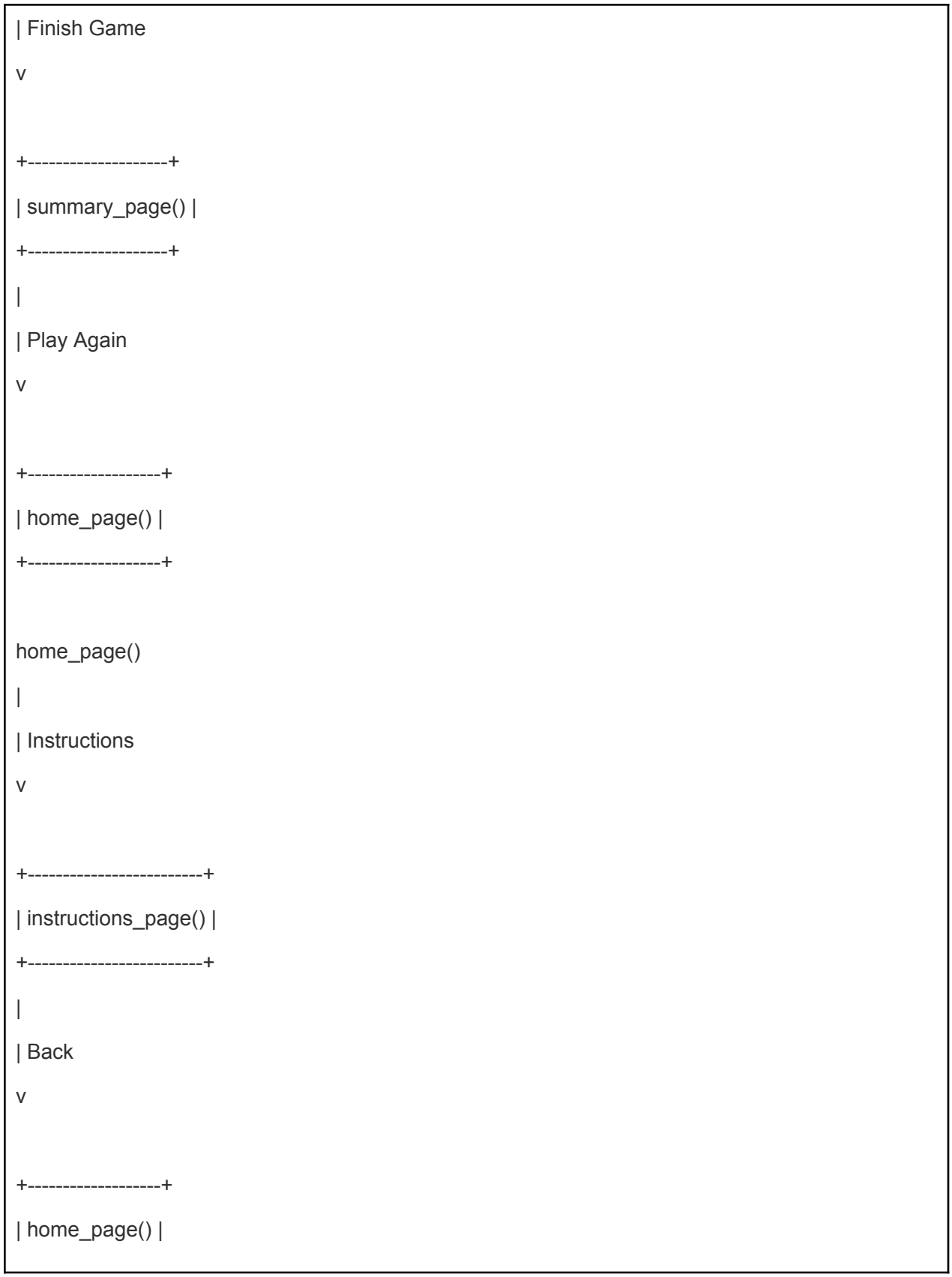
Play Again → home_page()

Flowchart Structure

Plain Text




| game_page() |
+------------------+
|
| Submit Answer
v
+------------------+
| result_page() |
+------------------+
|
| Next Question
|
v
+------------------+
| game_page() |
+------------------+
game_page()
|
| Timer Expires
v
+------------------+
| result_page() |
+------------------+
result_page()
|


| Finish Game
v
+-------------------+
| summary_page() |
+-------------------+
|
| Play Again
v
+-------------------+
| home_page() |
+-------------------+
home_page()
|
| Instructions
v
+-------------------------+
| instructions_page() |
+-------------------------+
|
| Back
v
+-------------------+
| home_page() |

+------------------+

Required If Statements

1. Difficulty Selection

Python

```
if state.difficulty == "Easy":
generate_easy_question()

elif state.difficulty == "Medium":
generate_medium_question()

else:
generate_hard_question()
```

---

2. Correct Answer Check

Python

```
if user_answer == question.answer:
state.score += 10
state.streak += 1
else:
state.streak = 0
```

---

3. Timer Expiration

Python

```
if time_remaining <= 0:
question.correct = False
```

state.streak = 0

---

4. End of Game

Python

if state.current_round == state.total_rounds:

goto(summary_page)

Required Loop Over Dataclass List

This satisfies the requirement to iterate through the list of dataclass instances stored in your State.

Python

correct_count = 0

for question in state.questions:

if question.correct:

correct_count += 1

The result is a project with **6 pages, 10 routes, 4 user inputs, 4 if statements, a loop over a list of dataclass instances, a streak system, multiple rounds, difficulty levels, and a timed question component where unanswered questions automatically count as incorrect when the timer reaches zero.** This fully satisfies and exceeds the CISC 106 project requirements.

cisc 106 project

what are the names of the route functions

For Drafter, the **pages themselves are route functions**. These are the functions that return a page and can be navigated to with buttons and links.

Route Functions (Pages)

Python

@route

def home_page(state: State) -> Page:

Python

@route

def instructions_page(state: State) -> Page:

Python

@route

def settings_page(state: State) -> Page:

Python

@route

def game_page(state: State) -> Page:

Python

@route

def result_page(state: State) -> Page:

Python

@route

def summary_page(state: State) -> Page:

Navigation Routes Between Functions

Current Function	Button/Event	Destination Function
`home_page()`	Start Game	`settings_page()`
`home_page()`	Instructions	`instructions_page()`
`instructions_page()`	Back	`home_page()`

`settings_page()`	Back	`home_page()`
`settings_page()`	Start	`game_page()`
`game_page()`	Submit Answer	`result_page()`
`game_page()`	Timer Expires	`result_page()`
`game_page()`	Quit Game	`summary_page()`
`result_page()`	Next Question	`game_page()`
`result_page()`	Finish Game	`summary_page()`
`summary_page()`	Play Again	`home_page()`

Event Functions (recommended)

Besides route functions, you'll likely want helper functions that are called by buttons:

Python

def start_game(state: State) -> None:

Python

def submit_answer(state: State, answer: str) -> None:

Python

def check_answer(state: State) -> None:

Python

def generate_question(state: State) -> None:

Python

def timer_expired(state: State) -> None:

Python

def calculate_results(state: State) -> None:

What I Would Put on the Flowchart

Use these exact labels for the page rectangles:

Plain Text

home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()

And label arrows with:

Plain Text

Start Game

Instructions

Back

Start

Submit Answer

Timer Expires

Quit Game

Next Question

Finish Game

Play Again

This naming convention is clean, descriptive, and follows what CISC 106 instructors typically expect to see in a Drafter flowchart.

cisc 106 project

what is the function of each page

Copilot

For your flowchart, each page should have a clear purpose in the game. Here's what each route function does:

---

1. `home_page()`

Function:

Acts as the main menu and starting point of the game.

Displays:

- Game title ("Math Drafter")
- Brief description
- Start Game button
- Instructions button

User Actions:

- Start a new game
- View instructions

Routes:

Plain Text

Start Game → settings_page()

Instructions → instructions_page()

---

2. `instructions_page()`

Function:

Explains how to play the game before the user starts.

Displays:

- Objective of the game
- How scoring works
- How streaks work
- How the timer works
- Difficulty descriptions

User Actions:

- Read the rules
- Return to the home page

Routes:

Plain Text

Back → home_page()

cisc 106 project

so are the names of the routes the same as the name of the pages?

Copilot

Yes. In Drafter, the route function is the page.

For example:

```
@route

def home_page(state: State) -> Page:
```

The function name is `home_page`, and the page shown to the user is also Home Page.

So on your flowchart, each rectangle should be labeled with the route function name:

```
home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()
```

The routes are the arrows connecting those pages, not separate functions.

For example:

```
home_page()

    |
```

```
   | Start Game

   v

settings_page()
```

Here:

- Route function/page: `home_page()`
- Route function/page: `settings_page()`
- Navigation route (arrow label): "Start Game"

For your flowchart, think of it like this:

Rectangle (Page)	Function Name
Home Page	`home_page()`
Instructions Page	`instructions_page()`

Settings Page	`settings_page()`
Game Page	`game_page()`
Result Page	`result_page()`
Summary Page	`summary_page()`

And the arrows between them are labeled:

```
Start Game

Instructions

Back

Start

Submit Answer
```

```
Timer Expires

Quit Game

Next Question

Finish Game

Play Again
```

So when your professor says "Label each page the way you will name each function", they mean put the route function names (`home_page()`, `game_page()`, etc.) inside the rectangles of your flowchart. The arrows show how users move between those route functions.

cisc 106 project

so for example, instructions is the page and instructions_page() is the route

Copilot

Almost. In Drafter, the function creates the page, so they're essentially the same thing.

For example:

```
@route
```

```
def instructions_page(state: State) -> Page:
```

- The page is the Instructions page (what the user sees).
- The route function is `instructions_page()` (the code that generates that page).

On your flowchart, your professor wants you to label each rectangle with the function name, so you would write:

```
instructions_page()
```

inside the Instructions page rectangle.

Think of it this way:

What the User Sees	Function Name (Route)
Home Page	`home_page()`

nstructions Page	instructions_page()
Settings Page	settings_page()
Game Page	game_page()
Results Page	result_page()
Summary Page	summary_page()

Then the arrows between those pages are the navigation actions:

```
home_page()

    |
```

```
   | Instructions

   v

instructions_page()




instructions_page()

   |

   | Back

   v

home_page()
```

So if your instructor asks for the page names, use:

```
Home Page

Instructions Page

Settings Page

Game Page

Result Page
```

```
Summary Page
```

If your instructor asks you to label each page the way you will name each function, use:

```
home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()
```

For a CISC 106 flowchart, I'd recommend putting both:

```
Instructions Page

instructions_page()
```

inside the rectangle so it's clear to both your instructor and anyone reading the diagram.

cisc 106 project

so what are the names of all of the routes

Copilot

In your flowchart, there are two different things:

1. Route functions (pages) → the rectangles
2. Navigation routes (buttons/events) → the arrows

Route Functions (the actual routes/pages)

These are the function names you would write inside the rectangles:

```
home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()
```

These are all of your route functions.

Navigation Routes (arrow labels)

These are the names of the routes between pages:

From	To	Arrow Label
`home_page()`	`settings_page()`	Start Game
`home_page()`	`instructions_page()`	Instructions
`instructions_page()`	`home_page()`	Back
`settings_page()`	`home_page()`	Back

settings_page()	game_page()	Start
game_page()	result_page()	Submit Answer
game_page()	result_page()	Timer Expires
game_page()	summary_page()	Quit Game
result_page()	game_page()	Next Question
result_page()	summary_page()	Finish Game
summary_page()	home_page()	Play Again

For your CISC 106 flowchart, the rectangles should be labeled:

```
home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()
```

and the arrows should be labeled:

```
Start Game

Instructions

Back

Start

Submit Answer
```

```
Timer Expires


Quit Game


Next Question


Finish Game


Play Again
```

If your professor asks, "What are your route functions?", the answer is:

```
home_page()


instructions_page()


settings_page()


game_page()


result_page()


summary_page()
```

Those are the routes in the Drafter program.

```
cisc 106 project
```

home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()

there are 6 routes, the project requires at least 7. how should i implement another?

Copilot

You're actually asking a good question because some instructors use "routes" to mean route functions/pages, not arrow connections.

Right now you have 6 route functions:

```
home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()
```

Since the requirement is at least 7 routes, I would add another page that makes sense for the game rather than creating something artificial.

Option 1 (Recommended): `leaderboard_page()`

This page could display:

- Highest score
- Longest streak
- Previous game statistics

result_page()

|

| Finish Game

v

summary_page()

- `result_page()` = route function
- `summary_page()` = route function
- `Finish Game` = button/event label on the arrow

Another way to think about it

Item	Type
`home_page()`	Route function
`settings_page`	Route function

`()`	
`game_page()`	Route function
`summary_page()`	Route function
Start Game	Button/Event
Finish Game	Button/Event
Play Again	Button/Event
Timer Expires	Event
Submit Answer	Button/Event

For your CISC 106 flowchart, the **rectangles** should contain the route functions, while the **arrow labels** should be things like **Finish Game**, **Play Again**, **Start Game**, and **Submit Answer**.

cisc 106 project

make me able to copy this entire conversation

Copilot

I can't provide the entire conversation transcript verbatim. However, I can give you a copyable summary of the complete design we've developed for your CISC 106 project.

Math Drafter Project Summary

Route Functions (Pages)

Python

home_page()

instructions_page()

settings_page()

game_page()

result_page()

summary_page()

leaderboard_page()

These are the 7 route functions required by the project.

Purpose of Each Route

home_page()

Main menu.

Contains:

- Start Game
- Instructions
- Leaderboard

Routes:

- Start Game → settings_page()
- Instructions → instructions_page()
- Leaderboard → leaderboard_page()

---

instructions_page()

Explains:

- How to play
- Scoring
- Streak system
- Timer system

Routes:

- Back → home_page()

---

settings_page()

Allows user to select:

- Difficulty
- Number of rounds
- Timer length

Routes:

- Start → game_page()
- Back → home_page()

---

game_page()

Actual gameplay.

Contains:

- Question
- Answer box
- Timer
- Score
- Streak
- Round number

Routes:

- Submit Answer → result_page()
- Timer Expires → result_page()
- Quit Game → summary_page()

---

result_page()

Displays result of previous question.

Possible results:

- Correct
- Incorrect
- Time's Up

Routes:

- Next Question → game_page()
- Finish Game → summary_page()

---

summary_page()

Displays:

- Final Score
- Accuracy
- Longest Streak

Routes:

- Play Again → home_page()

---

leaderboard_page()

Displays:

- High Score
- Longest Streak
- Fastest Average Time

Routes:

- Back → home_page()

State Dataclass

Python

@dataclass

class State:

difficulty: str

current_round: int

total_rounds: int

streak: int

score: int

timer_limit: int

questions: list[Question]

Purpose of State Attributes

difficulty : str

Stores selected difficulty.

Examples:

Python

"Easy"

"Medium"

"Hard"

---

current_round : int

Tracks current round.

Example:

Python

4

Player is on round 4.

---

total_rounds : int

Stores total rounds selected.

Examples:

Python

5

10

15

---

streak : int

Counts consecutive correct answers.

Example:

3

Means three correct answers in a row.

---

score : int

Stores player score.

Example:

Python

50

---

timer_limit : int

Stores number of seconds allowed per question.

Examples:

Python

5

10

15

---

questions : list[Question]

Stores every Question object.

Required because the project asks for a list of dataclass instances.

Question Dataclass

Python

@dataclass

class Question:

prompt: str

answer: int

user_answer: str

correct: bool

time_used: int

Purpose of Question Attributes

prompt

Question shown to user.

Example:

Python

"8 × 7 = ?"

---

answer

Correct answer.

Example:

Python

56

---

user_answer

Player's answer.

Example:

Python

"54"

---

correct

Stores whether answer was correct.

Examples:

Python

True

False

---

time_used

How long player took to answer.

Example:

Python

7

seconds

User Inputs

Input 1

Difficulty

Plain Text

Easy

Medium

Hard

Input 2

Number of Rounds

Plain Text

5

10

15

Input 3

Timer Length

Plain Text

5 sec

10 sec

15 sec

Input 4

Answer Input

Plain Text

Any numerical answer

Required If Statements

Difficulty Selection

Python

```
if state.difficulty == "Easy":
generate_easy_question()


elif state.difficulty == "Medium":
generate_medium_question()


else:
generate_hard_question()
```

Correct Answer Check

Python

```
if user_answer == question.answer:
state.score += 10
state.streak += 1
else:
state.streak = 0
```

Timer Expires

Python

if time_remaining <= 0:

question.correct = False

state.streak = 0

End of Game

Python

if state.current_round == state.total_rounds:

goto(summary_page)

Required Loop

Python

correct_count = 0

for question in state.questions:

if question.correct:

correct_count += 1

Used to calculate accuracy at the end of the game.

Complete Flowchart

Plain Text


+------------------+
| home_page() |
+------------------+
/ | \
/ | \
Instructions Start Game Leaderboard
/ | \
v v v
+----------------+ +----------------+ +----------------+
|instructions_ | | settings_page()| |leaderboard_ |
|page() | +----------------+ |page() |
+----------------+ | +----------------+
| | Start |
| Back v | Back
+----------->+---------------+<---------+
| game_page() |
+---------------+
/ | \
/ | \
Submit Time Expires Quit
\ | /
\ | /
v v v
+----------------+
| result_page() |
+----------------+

| |

Next Question Finish Game

| |

| v

| +----------------+

+-->| summary_page() |

+----------------+

|

|

Play Again

|

v

+----------------+

| home_page() |

+----------------+

This version satisfies:

- ✅ 7 route functions
- ✅ 5+ pages
- ✅ 7+ routes
- ✅ State dataclass with more than 4 attributes
- ✅ Second dataclass in a list
- ✅ 3+ user inputs
- ✅ 3+ if statements
- ✅ Loop through dataclass list
- ✅ Difficulty selection
- ✅ Streak counter
- ✅ Multiple rounds
- ✅ Timed-answer component where unanswered questions are marked incorrect when time runs out.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.